



Temporal Hypergraph Learning

How can the model learn temporal and higher-order interactions?

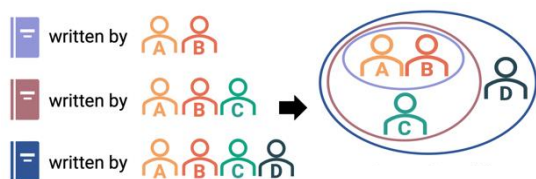
SuYong Jeong
Machine Intelligence and Data Science Laboratory

Outline

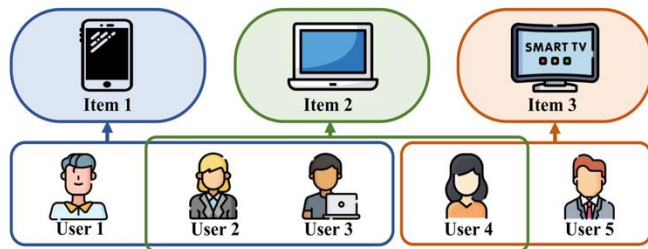
- ☐ About Hypergraphs
- ☐ Temporal Hypergraphs
- ☐ CAT-WALKS
- ☐ FastHeP

Hypergraphs

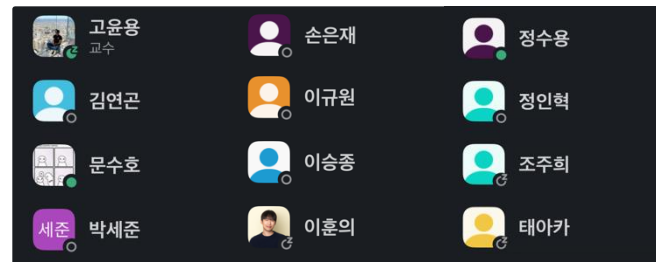
- Natural representation of higher-order interactions
- More expressive than pairwise graphs
- Real-world applications: co-authorship, co-purchase networks, and MINDS chatroom



Co-authorship



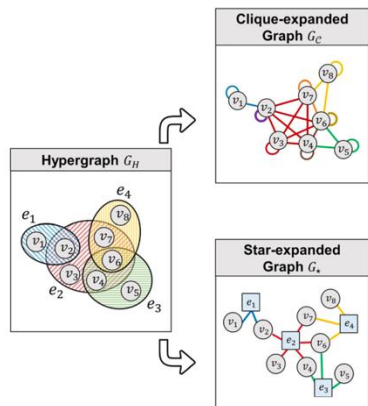
Co-purchase of items



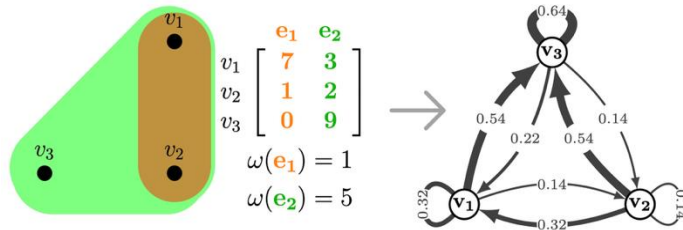
MINDS chatroom

Various Hypergraph Modeling Approaches

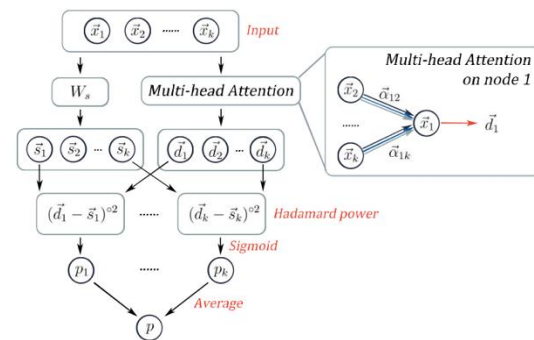
- Clique/star expansion
- Directed graph modeling with hyperedge weights and EDVW (Edge-dependent vertex weights)
- Direct modeling of hyperedges without decomposition



Clique/Star Expansion



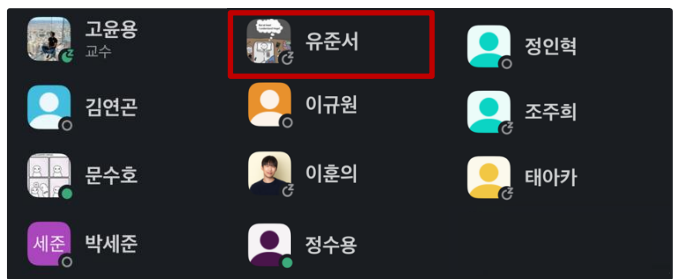
Representative Directed graph (2020)



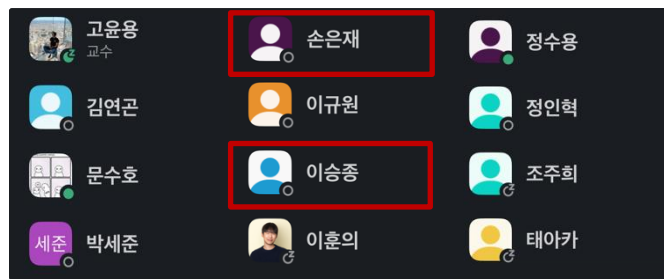
HyperSAGNN (2020)

Limitations of Hypergraphs – Dynamic Features

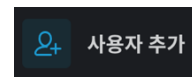
- ❑ Static hypergraph modeling capturing only the final state of hypergraphs
- ❑ Continuous network evolution over time
 - Failure to capture temporal evolutions in hypergraphs
 - Loss of temporal order among higher-order interactions



2025 Fall



2026 Spring (current)

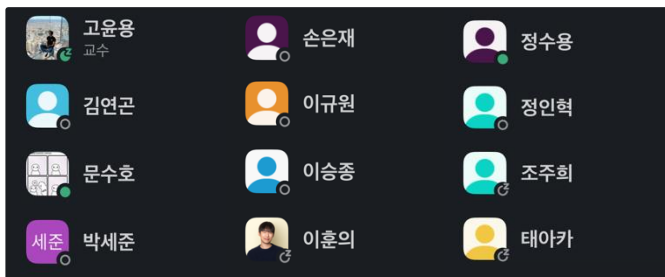


???

Temporal Hypergraph Learning

- Temporal hypergraph, a sequence of hyperedges with timestamps
- Modeling time-dependent, higher-order interactions
- Predicting future higher-order interactions

Simple Static Hypergraph



Temporal Hypergraph

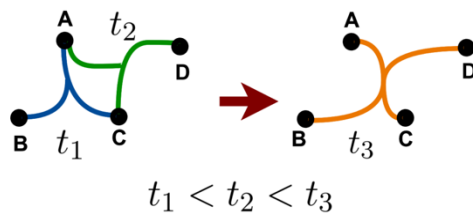
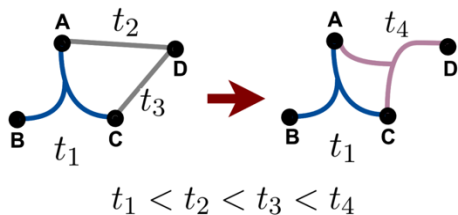


Upcoming Topics

- Two Models to represent complex higher-order relationships with temporal features
 - Considering both structural and temporal information

- CAAt-Walk: Causal anonymous set walks
 - A hyperedge-centric random walk on hypergraphs, SetWalk
 - Learning the underlying dynamic laws that govern the temporal and structural processes

- FastHeP: Fast and accurate approach for temporal hyperedge prediction
 - A hybrid message passing method
 - Strong expressiveness, high time & memory efficiency, and good scalability





CAt-Walk: Inductive Hypergraph Learning via Set Walks

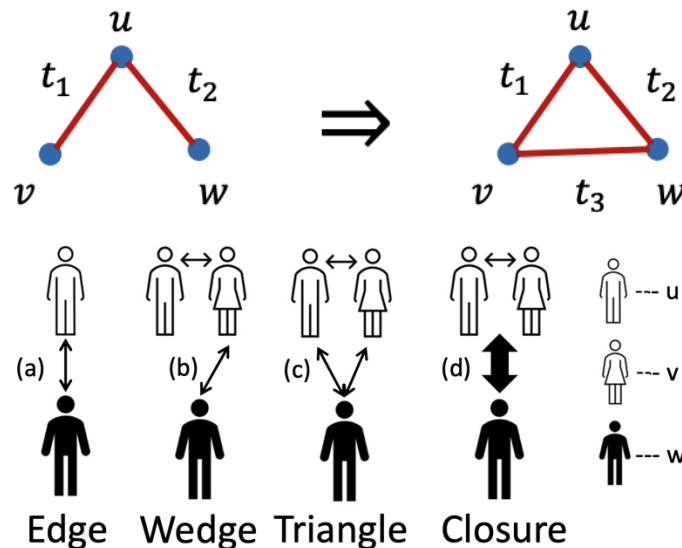
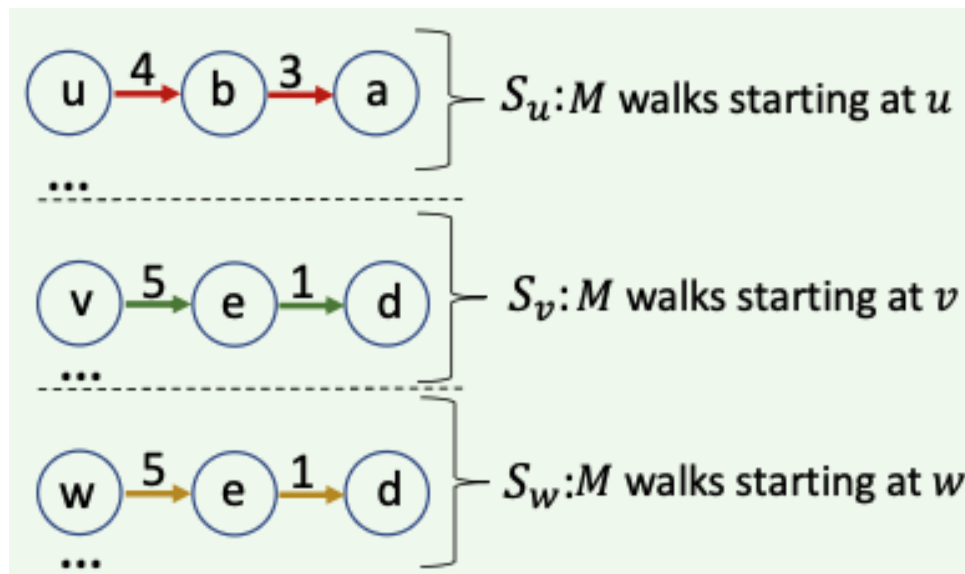
Ali Behrouz, Farnoosh Hashemi, Sadaf Sadeghian, Margo Seltzer

University of British Columbia

NeurIPS '23

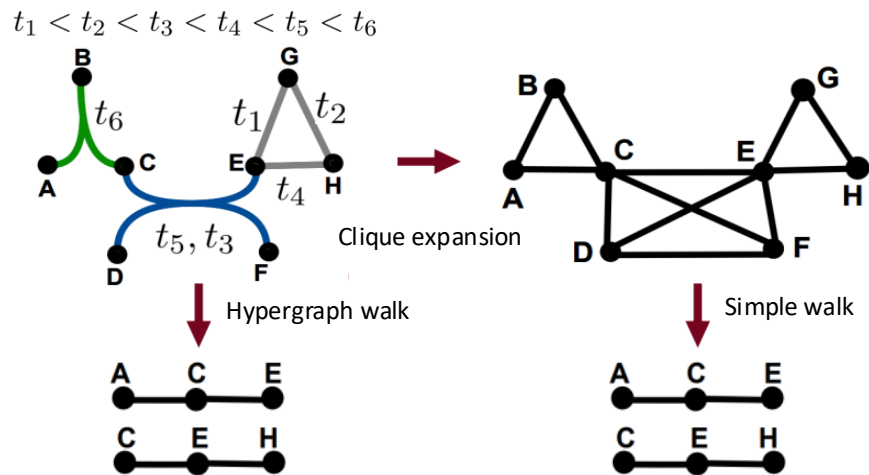
Existing Temporal Hypergraph Model - HIT

- Using **temporal random walk** to extract temporal motifs
- Prediction of underlying network laws via temporal structural information learning
- Applicable only to **triplets** $\{u, v, w\}$



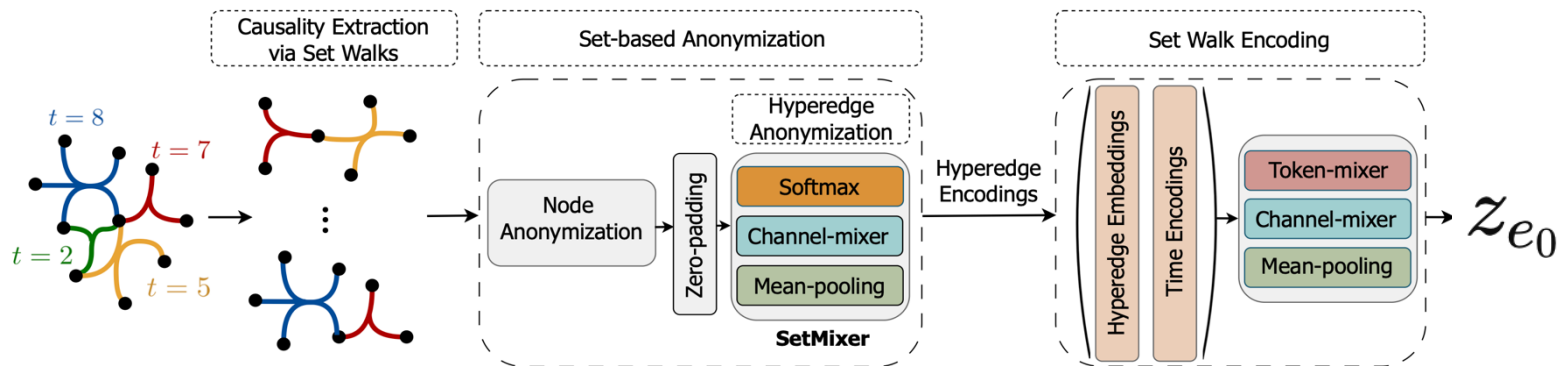
Problems of Existing Model

- ☐ Hyperedges are sets of nodes, but random walks are represented as sequences of nodes
- ☐ Equivalence to random walks on a weighted clique expansion
- ☐ **Loss of the semantic meaning of hyperedges**

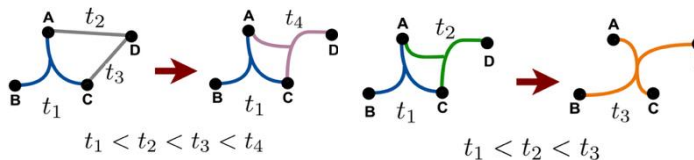


Methodology - Causal Anonymous Set Walks

- SetWalk -> a hyperedge-centric random walk on hypergraphs
- Learning temporal hypergraph motifs that reflect network dynamic laws

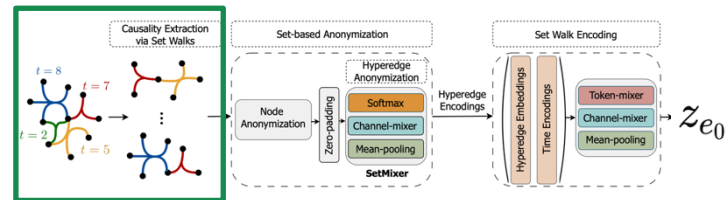


Network Dynamic Laws

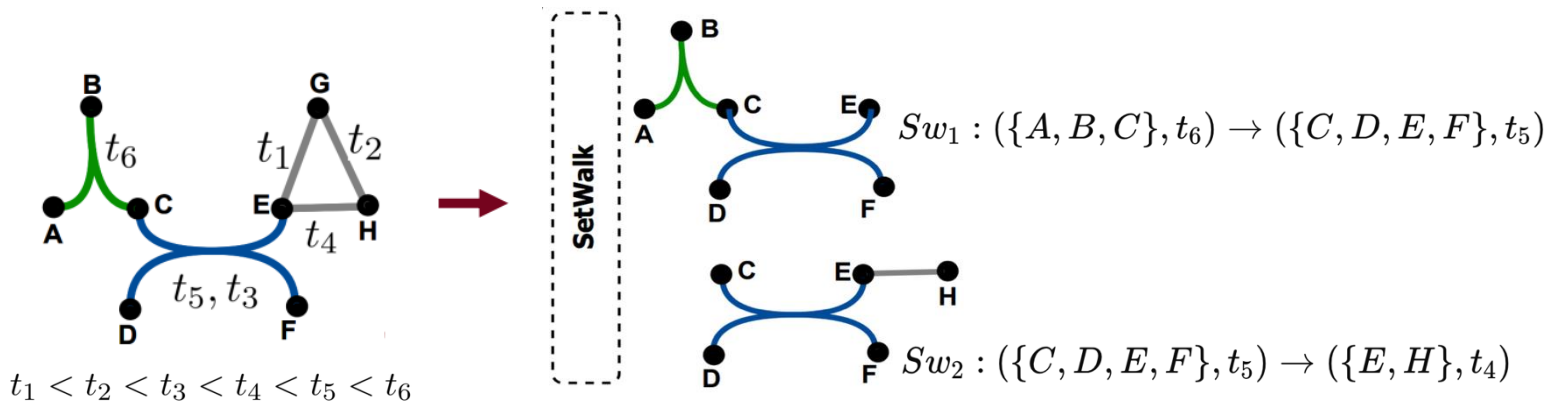


Causality Extraction via Set Walks

- Sequence of hyperedges generated by random walks
 - $Sw : (e_1, t_{e_1}) \rightarrow (e_2, t_{e_2}) \rightarrow \dots \rightarrow (e_\ell, t_{e_\ell})$
- Biased random walk with temporal bias and structural bias
 - Transitions toward structurally and temporally similar hyperedges



$$\mathbb{P}[(e, t)|(e_p, t_p)] \propto \frac{\exp(\alpha(t - t_p))}{\sum_{(e', t') \in \mathcal{E}^{tp}(e_p)} \exp(\alpha(t' - t_p))} \times \frac{\exp(\varphi(e, e_p))}{\sum_{(e', t') \in \mathcal{E}^{tp}(e_p)} \exp(\varphi(e', e_p))}$$

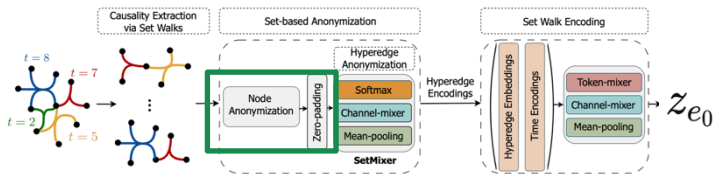


Set-based Anonymization of Hyperedges (cont.)

□ Position-wise counting of node occurrences in SetWalks

■ $e_0 = \{u_1, \dots, u_k\}$, node $w = \bigcup_{i=1}^k \mathcal{S}(u_i)$

■ $\mathcal{R}(w, \mathcal{S}(w_0))[i] = |\{\text{Sw} | \text{Sw} \in \mathcal{S}(w_0), w \in \text{Sw}[i][0]\}| \quad \forall i \in \{1, 2, \dots, m\}.$



Candidate hyperedge: $e_0 = \{A, B\}$

Set walks of node A: $S(A) = \{\text{Sw}_1, \text{Sw}_2\}$

$\text{Sw}_1 : (\{A, C\}, 10) \rightarrow (\{A, C, D\}, 8) \rightarrow (\{C, D\}, 5)$

$\text{Sw}_2 : (\{A, B\}, 9) \rightarrow (\{B, C\}, 7) \rightarrow (\{B, D\}, 4)$

Set walks of node B: $S(B) = \{\text{Sw}_3, \text{Sw}_4\}$

$\text{Sw}_3 : (\{A, B\}, 11) \rightarrow (\{B, C\}, 8) \rightarrow (\{C, E\}, 6)$

$\text{Sw}_4 : (\{B, D\}, 10) \rightarrow (\{A, B, D\}, 7) \rightarrow (\{A, D\}, 3)$

All nodes appeared in Set Walks

$$\begin{matrix} \{A, B, C, D, E\} & \rightarrow & R(A, S(A)) = [2, 1, 0] & & R(A, S(B)) = [1, 1, 1] \\ & & R(E, S(A)) = [0, 0, 0] & \cdots & R(E, S(B)) = [0, 0, 1] \end{matrix}$$

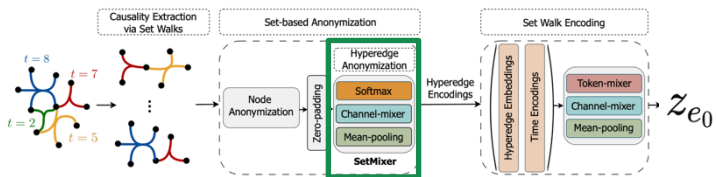
Node-dependent node identity

$$\text{ID}(w, e_0) = \{\mathcal{R}(w, \mathcal{S}(u_i))\}_{i=1}^k \rightarrow \begin{matrix} \text{Id}(A, e_0) = \{[2, 1, 0], [1, 1, 1]\} \\ \text{Id}(E, e_0) = \{\ddots, [0, 0, 1]\} \end{matrix}$$

Set-based Anonymization of Hyperedges

SetMixer

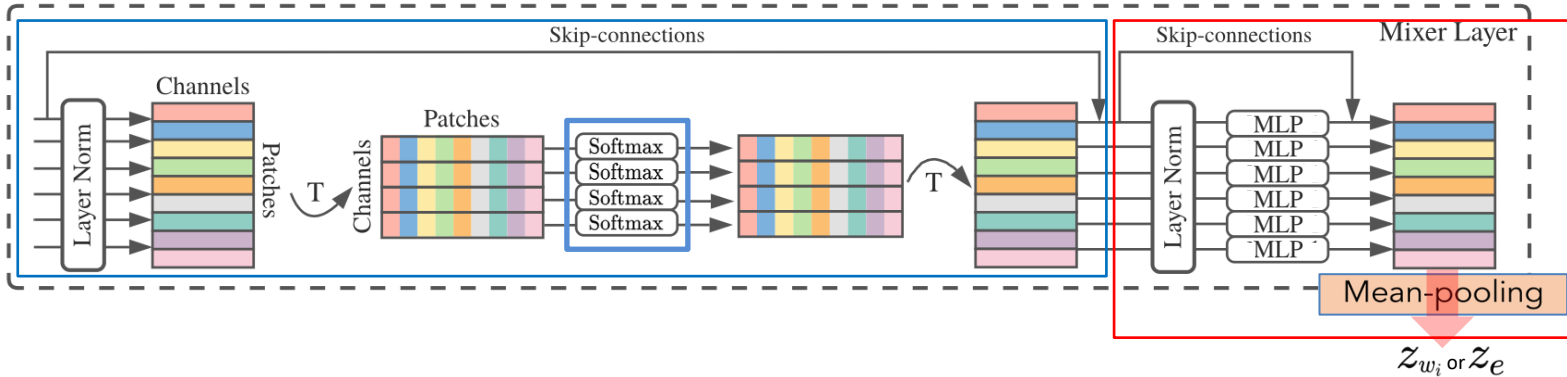
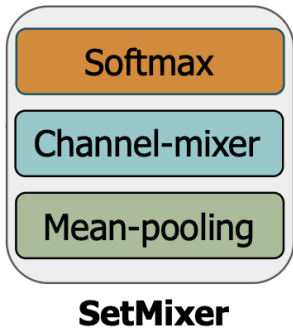
- Hyperedge anonymization using anonymized nodes
- Permutation-invariant pooling strategy**
- Representation of hyperedges in SetWalks with respect to an anchor hyperedge



$$\text{Id}(e, e_0) = \Psi_1 \left(\left\{ \Psi_2 (\text{Id}(w_i, e_0)) \right\}_{i=1}^{k'} \right) \Psi(.) : \mathbb{R}^{d \times d_1} \rightarrow \mathbb{R}^{d_2}$$

Hyperedge Anonymization

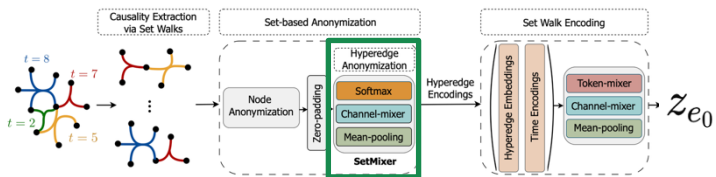
$$\mathbf{H}_{\text{token}} = \mathbf{V} + \sigma \left(\text{Softmax} \left(\text{LayerNorm}(\mathbf{V})^T \right) \right)^T \Psi(S) = \text{MEAN} \left(\mathbf{H}_{\text{token}} + \sigma \left(\text{LayerNorm}(\mathbf{H}_{\text{token}}) \mathbf{W}_s^{(1)} \right) \mathbf{W}_s^{(2)} \right)$$



Details of SetMixer (cont.)

$$SW_1 : (\{A, C\}, 10) \rightarrow (\{A, C, D\}, 8) \rightarrow (\{C, D\}, 5)$$

$$Id(e, e_0) = \Psi_1 \left(\underbrace{\{\Psi_2 (Id(w_i, e_0))\}_{i=1}^{k'}}_{\text{red arrow}} \right) \quad \Psi(.) : \mathbb{R}^{d \times d_1} \rightarrow \mathbb{R}^{d_2}$$



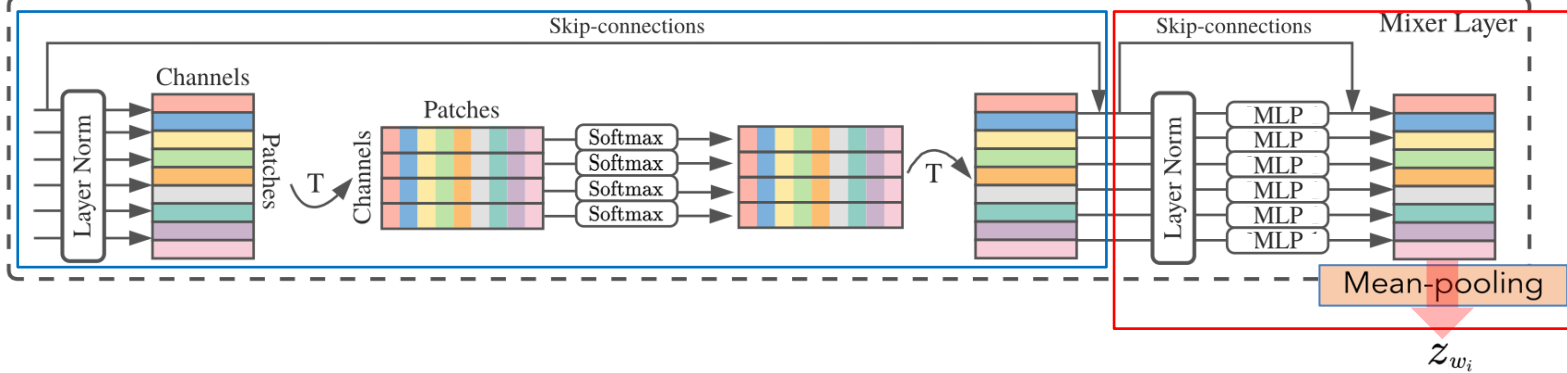
$$Id(A, e_0) \quad \mathbf{H}_{\text{token}} = \mathbf{V} + \sigma \left(\text{Softmax} \left(\text{LayerNorm}(\mathbf{V})^T \right) \right)^T \quad \Psi(S) = \text{MEAN} \left(\mathbf{H}_{\text{token}} + \sigma \left(\text{LayerNorm}(\mathbf{H}_{\text{token}}) \mathbf{W}_s^{(1)} \right) \mathbf{W}_s^{(2)} \right)$$

$$= \{[2, 1, 0], [1, 1, 1]\}$$

$$R(A, S(A)) \quad R(A, S(B))$$

$$V_A = \begin{bmatrix} 2 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

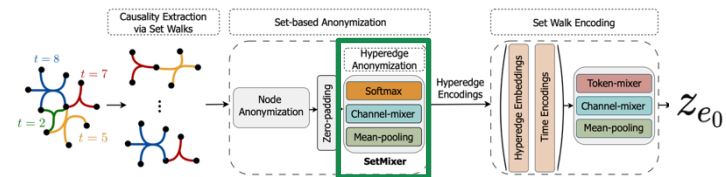
Zero-padding



Details of SetMixer

$$SW_1 : (\{A, C\}, 10) \rightarrow (\{A, C, D\}, 8) \rightarrow (\{C, D\}, 5)$$

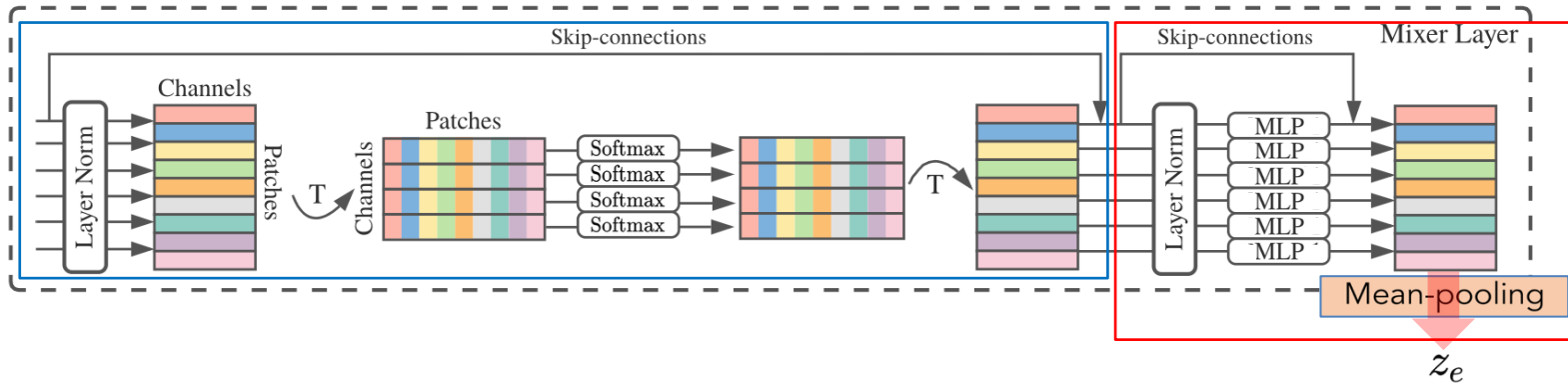
$$Id(e, e_0) = \Psi_1 \left(\{\Psi_2 (Id(w_i, e_0))\}_{i=1}^{k'} \right) \quad \Psi(.) : \mathbb{R}^{d \times d_1} \rightarrow \mathbb{R}^{d_2}$$



$$\mathbf{H}_{\text{token}} = \mathbf{V} + \sigma \left(\text{Softmax} \left(\text{LayerNorm}(\mathbf{V})^T \right) \right)^T \quad \Psi(S) = \text{MEAN} \left(\mathbf{H}_{\text{token}} + \sigma \left(\text{LayerNorm}(\mathbf{H}_{\text{token}}) \mathbf{W}_s^{(1)} \right) \mathbf{W}_s^{(2)} \right)$$

$$\begin{aligned} & \mathbf{z}_A \\ & \Psi_2(\{[2, 1, 0], [1, 1, 1]\}) \\ & \mathbf{z}_C \\ & \Psi_2(\{[1, 2, 1], [0, 1, 1]\}) \end{aligned}$$

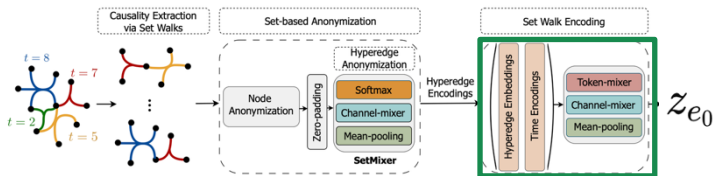
Zero-padding



SetWalk Encoding

□ Using RNN or Transformer in previous walk-based methods

- Computational cost
- Fail to capture long-term dependencies
- Causing over-squashing

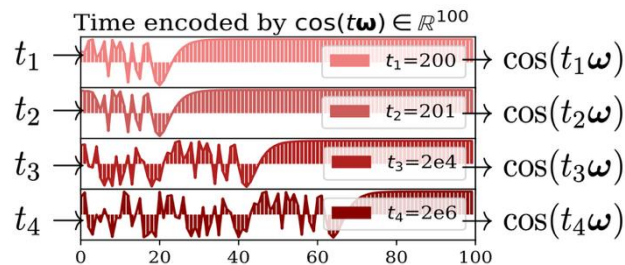


□ A time encoding module

$$\mathbf{T}(t) = (\omega_l t + \mathbf{b}_l) \parallel \cos(t\omega_p)$$

□ A mixer module

- Summarizing temporal and structural information extracted by SetWalks



$$\text{ENC}(\text{Sw}) = \text{MEAN} \left(\mathbf{H}_{\text{token}} + \sigma \left(\text{LayerNorm}(\mathbf{H}_{\text{token}}) \mathbf{W}_{\text{channel}}^{(1)} \right) \mathbf{W}_{\text{channel}}^{(2)} \right),$$

$$\text{where } \mathbf{H}_{\text{token}} = \mathbf{E} + \mathbf{W}_{\text{token}}^{(2)} \sigma \left(\mathbf{W}_{\text{token}}^{(1)} \text{LayerNorm}(\mathbf{E}) \right). \quad \mathbf{E}_i = \text{Id}(e_i, e_0) \parallel \mathbf{T}(t_{e_i})$$

Experimental Setup

- ☐ Baselines
 - Deep hypergraph learning methods (HyperSAGCN, NHP, CHESHIRE)
 - CE methods (CE-CAW, CE-EvolveGCN, CE-GCN)
- ☐ Ten datasets
- ☐ Downstream tasks
 - Hyperedge prediction
 - Node classification
- ☐ Transductive and inductive settings

Dataset	NDC Class	High School	Primary School	Congress Bill	Email Enron	Email Eu	Question Tags M	Users-Threads	NDC Substances	Question Tags U
$ V $	1,161	327	242	1,718	143	998	1,629	125,602	5,311	3,029
$ E $	49,724	172,035	106,879	260,851	10,883	234,760	822,059	192,947	112,405	271,233
#Timestamps	5,891	7,375	3,100	5,936	10,788	232,816	822,054	189,917	7,734	271,233
Task	HeP	HeP& NC	HeP & NC	HeP	HeP	HeP	HeP	HeP	HeP	HeP

Hyperedge Prediction



- Near perfect results in the transductive setting
- High scores in the inductive setting
- Email Enron?

	Datasets											
	NDC Class			High School			Primary School			Congress Bill		
	Metric	AUC	AP	AUC	AP	AUC	AP	AUC	AP	AUC	AP	AUC
Inductive	Strongly Inductive											
	CE-GCN	52.31 ± 2.99	54.33 ± 2.48	60.54 ± 2.06	59.92 ± 2.25	52.34 ± 2.75	56.41 ± 2.06	49.18 ± 3.61	53.85 ± 3.92	63.04 ± 1.80	57.70 ± 2.27	
	CE-EvolveGCN	49.78 ± 3.13	55.24 ± 3.56	46.12 ± 3.83	52.87 ± 3.48	58.01 ± 2.56	55.68 ± 2.41	54.00 ± 1.84	50.27 ± 1.76	57.31 ± 4.19	54.52 ± 3.79	
	CE-CAW	76.45 ± 0.29	78.58 ± 1.32	83.73 ± 1.42	82.96 ± 1.04	80.31 ± 1.46	82.84 ± 1.71	75.38 ± 1.25	77.19 ± 1.38	70.81 ± 1.13	72.07 ± 1.52	
	NHP	70.53 ± 4.95	68.18 ± 4.31	65.29 ± 3.80	62.86 ± 3.74	70.86 ± 3.42	71.31 ± 3.51	69.82 ± 1.27	64.09 ± 2.87	49.71 ± 6.09	50.01 ± 4.87	
	HyperSAGCN	79.05 ± 2.48	77.24 ± 2.05	88.12 ± 3.01	82.72 ± 2.93	80.13 ± 1.38	76.32 ± 2.96	79.51 ± 1.27	80.58 ± 2.61	73.09 ± 2.60	72.29 ± 3.69	
	CHESHIRE	72.24 ± 2.63	70.31 ± 2.26	82.54 ± 0.88	80.34 ± 1.19	77.26 ± 1.01	77.72 ± 0.76	79.43 ± 1.58	78.63 ± 1.25	70.03 ± 2.55	72.97 ± 1.81	
	CAI-WALK	98.89 ± 1.82	98.97 ± 1.69	96.03 ± 1.50	96.41 ± 0.70	95.32 ± 0.89	96.03 ± 0.84	93.54 ± 0.56	93.93 ± 0.36	73.45 ± 2.92	74.66 ± 3.87	
	Weakly Inductive											
	CE-GCN	51.80 ± 3.29	50.94 ± 3.77	50.33 ± 3.40	48.54 ± 3.92	52.19 ± 2.54	53.21 ± 3.59	52.38 ± 2.75	50.81 ± 2.68	50.81 ± 2.87	55.38 ± 2.79	
Transductive	CE-EvolveGCN	55.39 ± 5.16	57.24 ± 4.98	57.85 ± 3.51	63.26 ± 4.01	51.50 ± 4.07	52.59 ± 4.53	55.63 ± 3.41	51.19 ± 3.56	45.66 ± 2.10	50.93 ± 2.57	
	CE-CAW	77.61 ± 1.05	80.03 ± 1.65	83.77 ± 1.41	83.41 ± 1.19	82.98 ± 1.06	80.84 ± 1.57	79.51 ± 0.94	80.39 ± 1.07	80.54 ± 1.02	77.41 ± 1.28	
	NHP	75.17 ± 2.02	77.23 ± 3.11	67.25 ± 5.19	66.73 ± 4.94	71.92 ± 1.83	72.30 ± 1.89	69.58 ± 4.07	72.48 ± 4.83	60.38 ± 4.45	55.62 ± 4.67	
	HyperSAGCN	79.45 ± 2.18	80.32 ± 2.23	88.53 ± 1.26	87.26 ± 1.49	85.08 ± 1.45	86.84 ± 1.60	80.12 ± 2.00	73.48 ± 2.77	78.86 ± 0.63	79.14 ± 1.51	
	CHESHIRE	79.03 ± 1.24	78.98 ± 1.17	88.40 ± 1.06	86.53 ± 1.82	83.55 ± 1.27	79.42 ± 2.03	79.67 ± 0.83	80.03 ± 1.38	74.53 ± 0.91	75.88 ± 1.14	
	CAI-WALK	99.16 ± 1.08	99.33 ± 0.89	94.68 ± 2.37	96.54 ± 0.82	96.53 ± 1.39	96.83 ± 1.16	98.38 ± 0.21	98.48 ± 0.18	64.11 ± 7.96	67.68 ± 6.93	
	HPRA	70.83 ± 0.01	67.40 ± 0.00	94.91 ± 0.00	89.17 ± 0.00	89.86 ± 0.06	88.11 ± 0.02	79.48 ± 0.03	77.16 ± 0.03	78.62 ± 0.00	76.74 ± 0.00	
	HPLSF	76.19 ± 0.82	77.62 ± 1.42	92.14 ± 0.29	92.79 ± 0.15	88.57 ± 1.09	87.69 ± 1.61	79.31 ± 0.52	75.88 ± 0.43	75.73 ± 0.05	75.32 ± 0.08	
	CE-GCN	66.83 ± 3.74	65.83 ± 3.61	62.99 ± 3.02	59.76 ± 3.78	59.59 ± 3.46	69.42 ± 3.11	63.19 ± 3.34	58.06 ± 3.80	55.27 ± 3.12		
	CE-EvolveGCN	67.08 ± 3.51	66.51 ± 3.80	65.19 ± 2.26	59.27 ± 2.19	63.15 ± 1.32	65.18 ± 1.89	69.30 ± 2.27	64.38 ± 2.66	69.98 ± 3.58	67.76 ± 1.16	

	Datasets											
	Email Eu			Question Tags M			Users-Threads			NDC Substances		
	Metric	AUC	AP	AUC	AP	AUC	AUC	AP	AUC	AP	AUC	AP
Inductive	Strongly Inductive											
	CE-GCN	52.76 ± 2.41	50.37 ± 2.59	56.10 ± 1.88	54.15 ± 1.94	57.91 ± 1.56	59.45 ± 1.21	55.70 ± 2.91	54.29 ± 2.78	51.97 ± 2.91	55.03 ± 2.72	
	CE-EvolveGCN	44.16 ± 1.27	49.15 ± 1.23	64.08 ± 2.75	60.64 ± 2.78	52.00 ± 2.32	52.69 ± 2.15	58.17 ± 2.24	57.35 ± 2.13	54.57 ± 2.25	57.16 ± 2.55	
	CE-CAW	72.99 ± 0.20	73.45 ± 0.68	70.14 ± 1.89	70.26 ± 1.77	73.12 ± 1.06	72.64 ± 1.18	75.87 ± 0.77	73.19 ± 0.86	74.21 ± 2.04	76.52 ± 2.06	
	NHP	65.35 ± 2.07	64.24 ± 1.61	68.23 ± 3.34	69.82 ± 3.41	71.83 ± 2.64	71.09 ± 2.83	70.43 ± 3.64	73.22 ± 3.03	72.52 ± 2.90	71.56 ± 2.26	
	HyperSAGCN	78.01 ± 1.24	80.04 ± 1.87	73.66 ± 1.95	73.98 ± 1.35	73.94 ± 2.57	72.97 ± 2.45	75.85 ± 2.21	73.24 ± 2.75	78.88 ± 2.69	77.53 ± 2.28	
	CHESHIRE	69.98 ± 2.71	70.10 ± 3.05	N/A	N/A	76.99 ± 2.82	74.03 ± 2.78	76.60 ± 2.19	74.91 ± 2.71	75.04 ± 3.39	75.46 ± 2.90	
	CAI-WALK	91.68 ± 2.78	91.75 ± 2.82	88.03 ± 3.38	88.46 ± 3.09	89.84 ± 6.02	91.58 ± 4.37	93.29 ± 1.55	94.26 ± 1.21	97.59 ± 2.21	97.71 ± 2.07	
	Weakly Inductive											
	CE-GCN	49.60 ± 3.96	55.01 ± 3.25	55.13 ± 2.76	51.48 ± 2.66	57.06 ± 3.16	58.37 ± 2.86	60.92 ± 2.81	55.93 ± 2.03	56.85 ± 2.73	57.19 ± 2.52	
Transductive	CE-EvolveGCN	52.44 ± 2.38	50.61 ± 2.32	61.79 ± 1.63	59.61 ± 1.12	55.81 ± 2.54	50.63 ± 2.46	58.48 ± 2.49	55.90 ± 2.51	54.10 ± 1.21	56.13 ± 2.32	
	CE-CAW	73.54 ± 1.19	74.10 ± 1.41	77.29 ± 0.86	77.67 ± 1.94	80.79 ± 0.82	81.88 ± 0.63	77.28 ± 1.30	79.24 ± 1.19	76.51 ± 1.26	77.17 ± 1.39	
	NHP	67.19 ± 4.33	66.53 ± 4.21	70.46 ± 3.52	65.66 ± 3.94	76.44 ± 1.90	75.23 ± 1.96	73.37 ± 3.51	70.62 ± 3.71	78.15 ± 4.41	79.64 ± 4.32	
	HyperSAGCN	77.26 ± 2.09	74.05 ± 2.12	78.15 ± 1.41	76.19 ± 1.53	75.38 ± 1.43	70.35 ± 1.63	80.82 ± 2.18	76.67 ± 2.06	74.22 ± 1.91	70.57 ± 1.02	
	CHESHIRE	77.31 ± 0.95	76.01 ± 0.98	N/A	N/A	81.27 ± 0.85	82.96 ± 1.41	80.68 ± 1.31	80.78 ± 1.13	77.60 ± 1.57	79.48 ± 1.79	
	CAI-WALK	91.98 ± 2.41	92.22 ± 2.40	90.28 ± 2.81	90.56 ± 2.62	97.15 ± 1.81	97.55 ± 1.49	95.65 ± 1.82	96.18 ± 1.52	98.11 ± 1.31	98.25 ± 1.13	
	HPRA	72.51 ± 0.00	71.08 ± 0.00	83.18 ± 0.00	80.12 ± 0.00	70.49 ± 0.02	72.83 ± 0.00	77.94 ± 0.01	75.78 ± 0.01	81.05 ± 0.00	81.71 ± 0.00	
	HPLSF	75.27 ± 0.31	77.95 ± 0.14	83.45 ± 0.93	82.29 ± 1.06	74.38 ± 1.11	73.81 ± 1.45	82.12 ± 0.71	84.51 ± 0.62	80.89 ± 1.51	75.62 ± 1.38	
	CE-GCN	64.19 ± 2.79	65.93 ± 2.52	55.18 ± 5.12	55.84 ± 4.53	62.78 ± 2.69	59.71 ± 2.25	63.08 ± 2.19	65.37 ± 2.48	66.79 ± 2.88	60.51 ± 2.26	
	CE-EvolveGCN	64.36 ± 4.17	66.98 ± 3.72	72.56 ± 1.72	69.38 ± 1.51	68.55 ± 2.26	67.86 ± 2.61	70.09 ± 3.42	66.37 ± 3.17	71.31 ± 2.92	70.36 ± 2.72	

Ablation Studies

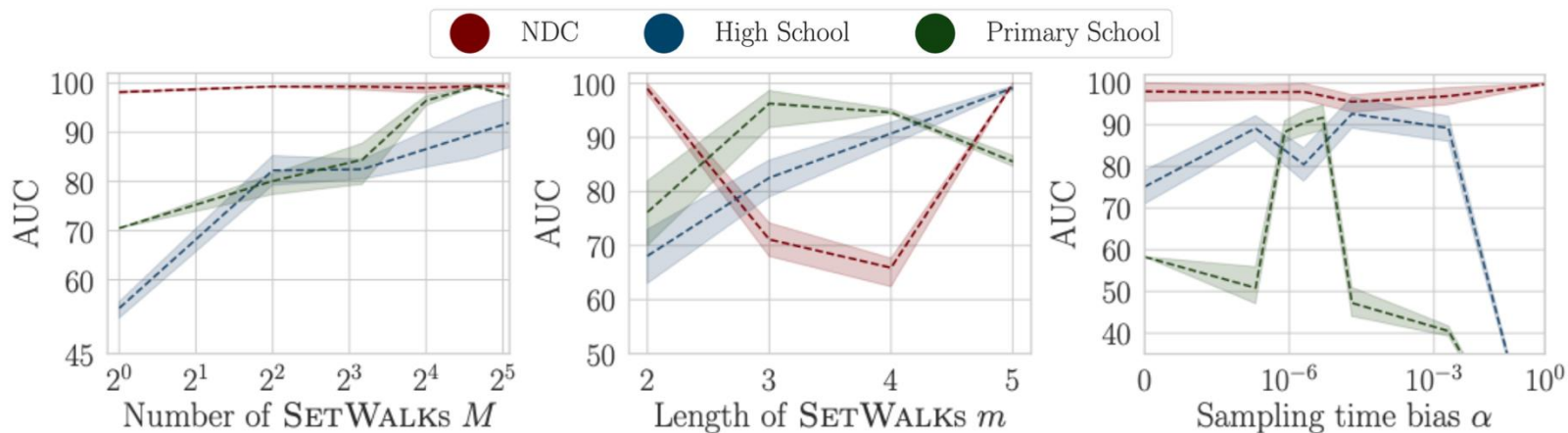
□ Each component is crucial for achieving superior performance

■ SetWalk, Set-Mixer pooling, etc.

Methods		High School	Primary School	Users in Threads	Congress bill	Question Tags U
1	CAT-WALK	96.03 ± 1.50	95.32 ± 0.89	89.84 ± 6.02	93.54 ± 0.56	97.59 ± 2.21
2	Replace SETWALK by Random Walk	92.10 ± 2.18	51.56 ± 5.63	53.24 ± 1.73	80.27 ± 0.02	67.74 ± 2.92
3	Remove Time Encoding	95.94 ± 0.19	86.80 ± 6.33	70.58 ± 9.32	92.56 ± 0.49	96.91 ± 1.89
4	Replace SETMIXER by MEAN(.)	94.58 ± 1.22	95.14 ± 4.36	63.59 ± 5.26	91.06 ± 0.24	68.62 ± 1.25
5	Replace SETMIXER by Sum-based	94.77 ± 0.67	90.86 ± 0.57	60.03 ± 1.16	91.07 ± 0.70	89.76 ± 0.45
6	Universal Approximator for Sets					
7	Replace MLP-MIXER by RNN	92.85 ± 1.53	50.29 ± 4.07	58.11 ± 1.60	54.90 ± 0.50	65.18 ± 1.99
8	Replace MLP-MIXER by Transformer	55.98 ± 0.83	86.64 ± 3.55	60.65 ± 1.56	89.38 ± 1.66	56.16 ± 4.03
9	Fix $\alpha = 0$	74.06 ± 14.9	58.3 ± 18.62	74.41 ± 10.69	93.31 ± 0.13	62.41 ± 4.34

Hyperparameter Sensitivity

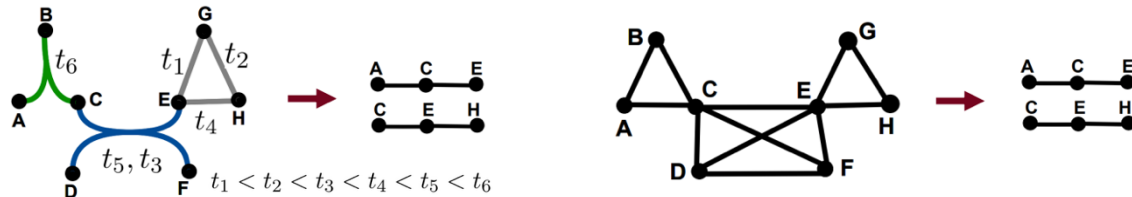
- Only a small number of sampled SetWalks are needed to achieve competitive performance
- Requirement for appropriate sampling time bias setting



Conclusion

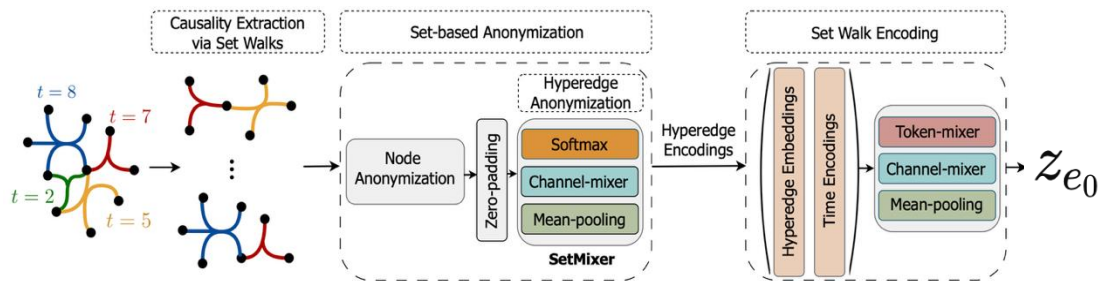
Existing Models

- Higher-order interactions information loss due to clique expansion



CAt-Walk

- A hyperedge-centric random walk on hypergraphs, SetWalk
- A permutation invariant pooling strategy, SetMixer





Fast and Accurate Temporal Hypergraph Representation for Hyperedge Prediction

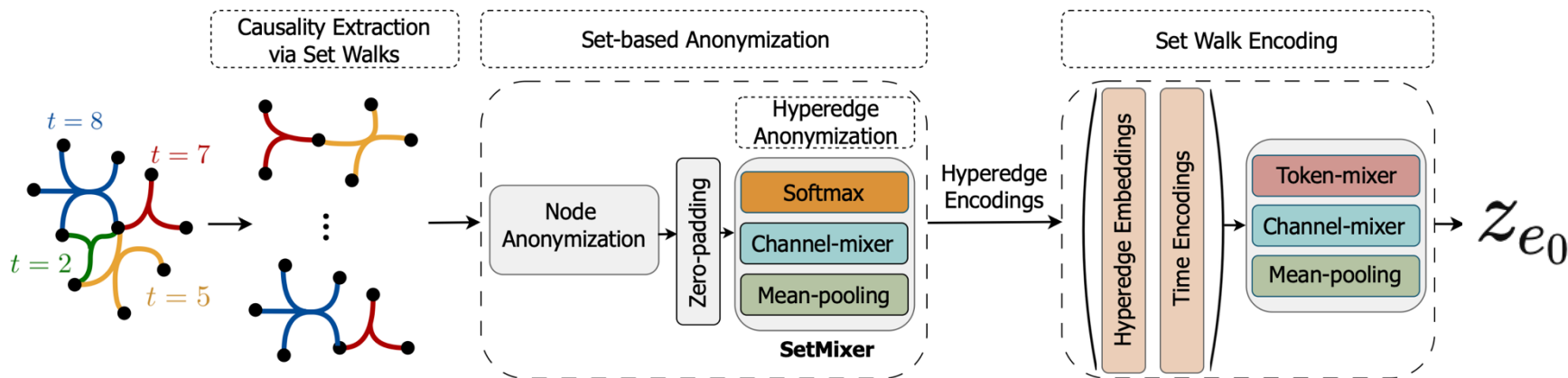
Yuanyuan Xu, Wenjie Zhang, Ying Zhang, Xiwei Xu, Xuemin Lin

¹University of New South Wales, ²Zhejiang Gongshang University, ³CSIRO Data61, ⁴Shanghai Jiao Tong University

KDD '25

Causal Anonymous Set Walks

- Extracting higher-order interactions with Set Walks, hyperedge-centric random walk
- SOTA temporal hypergraph approach



Computational Costs

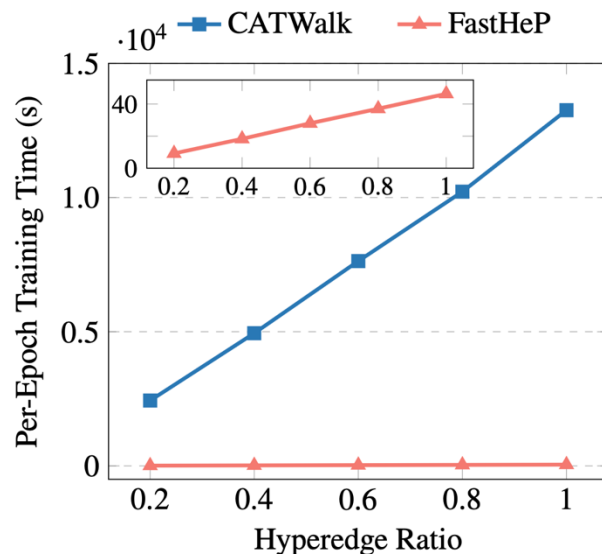
□ High time cost for computation graph construction in set walks

- Per-neighbor hyperedge computation cost $\rightarrow O(|H_{\mathcal{E}}|K^L)$
- Per-node computation cost $\rightarrow O(|H_{\mathcal{E}}|^2K^L)$
- Expensive storage costs $\rightarrow O(|V||\mathcal{E}|)$

□ Failing to process large temporal hypergraphs

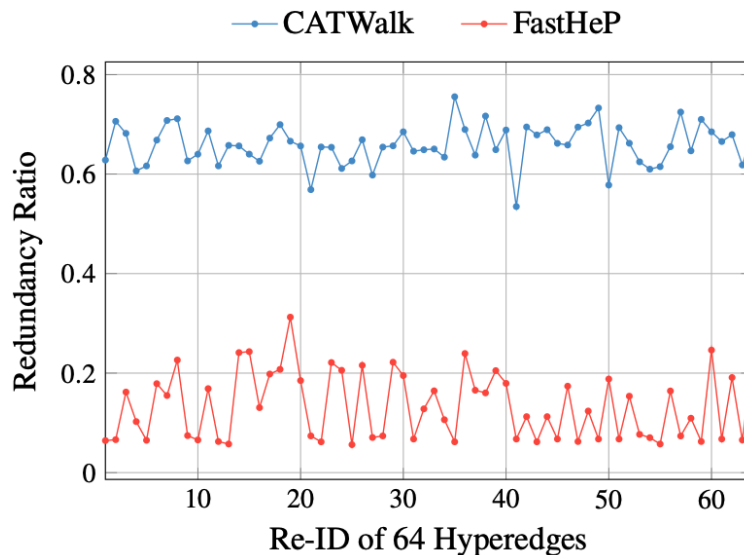
```
INFO:root:Transductive training...
INFO:module:neighbors: [6, 6, 6], pos dim: 172
INFO:root:num of training instances: 120442
INFO:root:num of batches per epoch: 1882
INFO:root:start 0 epoch
0% | 0/1882 [00:04<?, 7it/s]
Traceback (most recent call last):
  File "main.py", line 151, in <module>
    train_val(train_val_data, catn, args.mode, BATCH_SIZE, NUM_EPOCH, criterion, optimizer, early_stopper, ngh_finders, he_infos, rand_samplers,
  File "/home/dmais/SuYong/CATWalk/train.py", line 57, in train_val
    pos_prob, neg_prob = model.contrast(src_l_cut, src_l_fake, he_offset_l_cut, ts_l_cut) # the core training code
  File "/home/dmais/SuYong/CATWalk/module.py", line 120, in contrast
    neg_score = self.forward(neg_src_idx_l, he_offset_l, cut_time_l, subgraph_neg_src_idx_l, test=test)
  File "/home/dmais/SuYong/CATWalk/module.py", line 141, in forward
    encoded_hes = torch.cat((encoded_hes, encoded_he_batch), 0)
torch.cuda.OutOfMemoryError: CUDA out of memory. Tried to allocate 20.00 MiB (GPU 0; 15.70 GiB total capacity; 14.72 GiB already allocated; 20.5
> allocated memory try setting max_split size mb to avoid fragmentation. See documentation for Memory Management and PYTORCH_CUDA_ALLOC_CONF

INFO:root:Transductive training...
INFO:module:neighbors: [6, 6, 6], pos dim: 172
INFO:root:num of training instances: 120442
INFO:root:num of batches per epoch: 3764
INFO:root:start 0 epoch
100% | 3764/3764 [2:19:34<00:00, 2.22s/it]
```

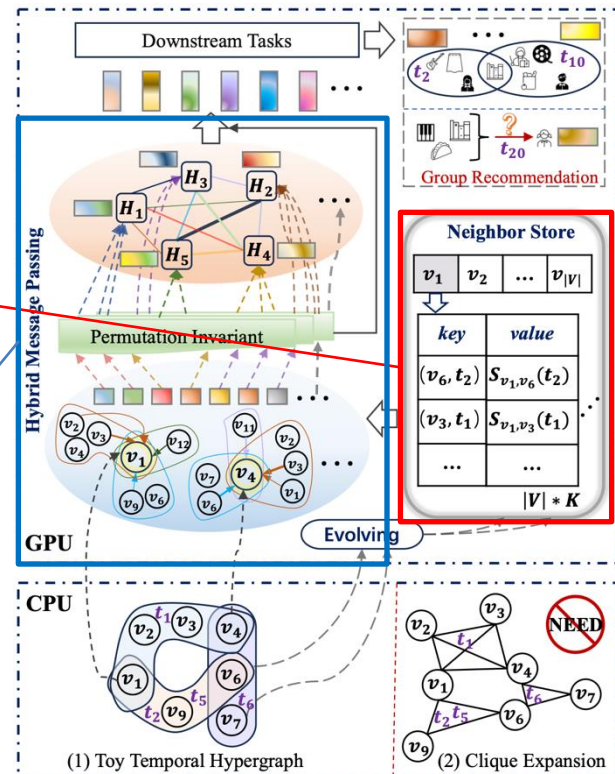


Computation Graph Analysis

- CAT-Walk exhibits a high rate of redundant neighbors, up to 76%
- Redundant neighbors dominating the hyperedge representation
 - Despite the diversity of neighbors in the computation graph
- **Over-smoothing** problem and sub-optimal embedding quality

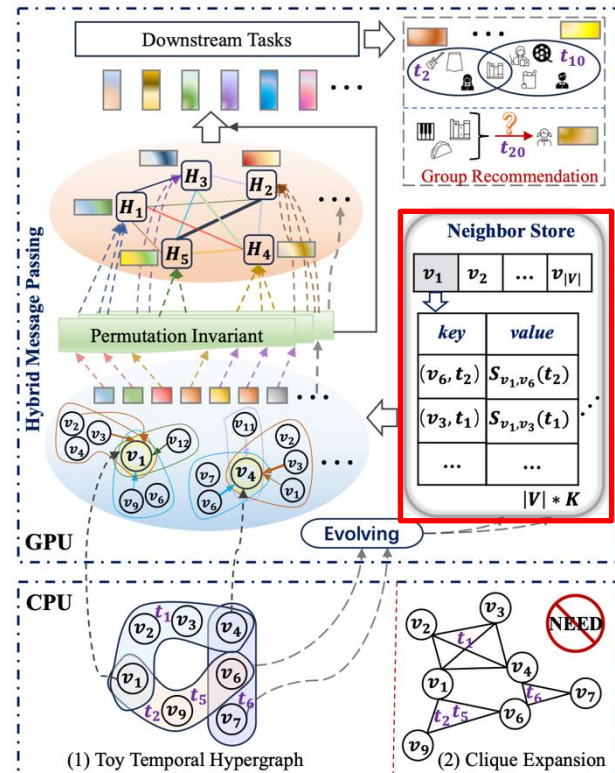


- Fast and accurate hypergraph representation approach for the temporal hyperedge prediction
 - $H_i \rightarrow \mathbf{h}_i(t_j) \in \mathbb{R}^d$
- An online hyperedge-centric neighbor store (HCNS)
 - No needs for an $O(|\mathcal{V}||\mathcal{E}|)$ incidence matrix
 - $O(|H_{\mathcal{E}}|K)$ -based local high-order structure capture
- Decoupling multi-hop hyperedge-centric message passing
 - Into hybrid node-wise and hyperedge-wise message passing



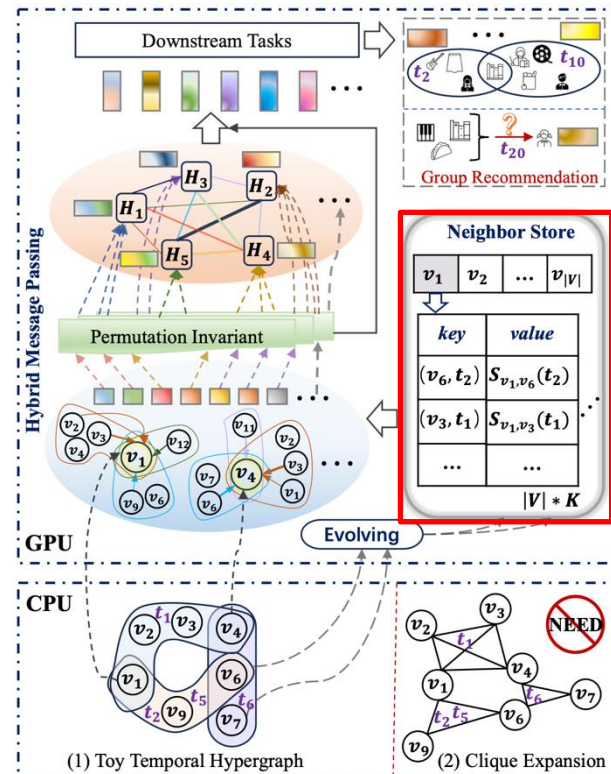
Hyperedge-Centric Neighbor Store, HCNS (cont.)

- $O(\overline{H_{\mathcal{E}}}|K)$ -based local high-order structure capture
 - No needs for an $O(|\mathcal{V}||\mathcal{E}|)$ incidence matrix
 - Storing a limited number of temporal neighbors for each node
 - $\mathcal{N}_v^K(t) : \{(v_i, t_j) \rightarrow s_{v,v_i}(t_j)\}, \exists \mathcal{V}_n, s.t., \{v, v_i\} \subseteq \mathcal{V}_n, n \leq |\mathcal{E}|, t_j \leq t.$
- Allocating a fixed-sized hash table
 - For each node v to store its K hyperedge-centric neighbors, $\mathcal{N}_v^K$
 - The hash function for the input (v_n, t_j) as $\text{hash}(v_n, t_j) = (\rho v_n) \bmod K$
 - A replacement rule based on θ , where $\theta \in [0.5, 1]$



Hyperedge-Centric Neighbor Store, HCNS

- $s_{v_i, v_n}(t)$ can store flexible values
 - Hyperedge features, node features, and time encoding features
 - Consuming at most $O(|\mathcal{V}|K(d_h + d_f + d_t))$ GPU memory
- $s_{v_i, v_n}(t) = \text{GRU}([s_{v_i, v_*}(t^-), \mathbf{e}(t), \phi(t - t^-), \mathbf{x}_{v_n}])$
 - Encoding the temporal dynamics of neighbors
- Similar to SetWalks in CAT-Walk



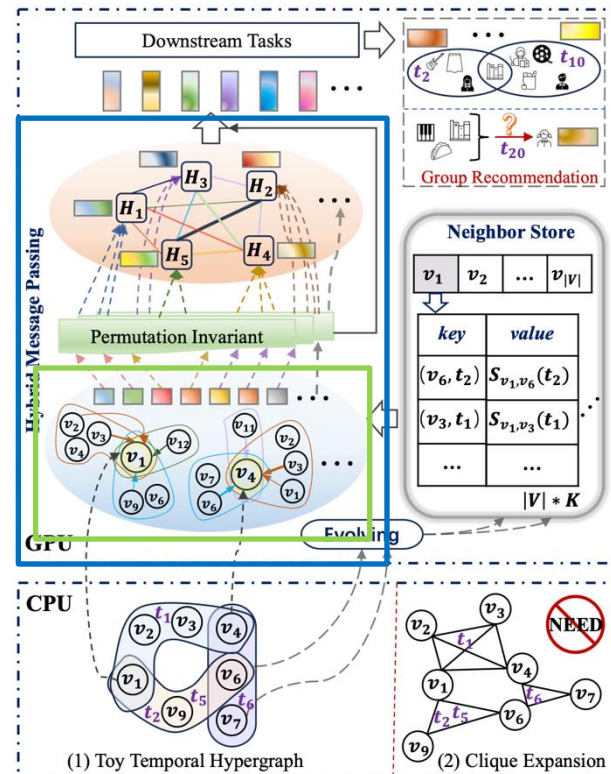
Temporal High-Order Structure Learning (cont.)

□ A novel hybrid message passing (blue box)

- Node-wise
- Hyperedge-wise message passing

□ Node-wise message passing (green box)

- $z_i(t) = \text{MLP}(z_i(t^-) \parallel \text{GAT}(q(t), \kappa(t), \nu(t)))$,
- $q(t) = z_i(t^-), \kappa(t) = \nu(t) = \{s_{v_i, v_n}(t_j) \mid \forall (v_n, t_j) \in \mathcal{N}_{v_i}^K(t)\}$
- Local high-order structure information and temporal dynamics



Temporal High-Order Structure Learning

□ Hyperedge-wise message passing (green box)

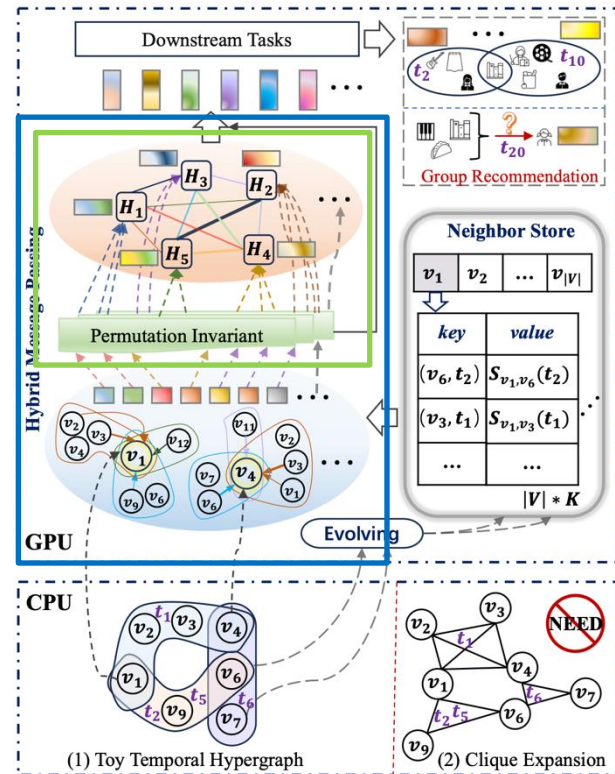
$$\blacksquare \hat{h}_j(t) = \frac{1}{|H_j|} \sum_{i=1}^{|H_j|} \text{MLP}(z_i(t)), \forall j \in \{1, \dots, |B|\}, \quad (6)$$

- $\hat{h}_j(t)$ in Eq. (6) is the j -th hyperedge features of $\hat{H}(t)$
- Permutation invariant with mean-pooling
- Local higher-order structure

$$\blacksquare H(t) = \hat{H}(t) + \text{ATT}(\mathbf{q}_h(t), \boldsymbol{\kappa}_h(t), \mathbf{v}_h(t)), \quad (7)$$

$$\mathbf{q}_h(t) = \boldsymbol{\kappa}_h(t) = \mathbf{v}_h(t) = \hat{H}(t). \quad (8)$$

- Global correlation learning among hyperedges using $\text{ATT}(\cdot)$
- Using both local and global high-order structures



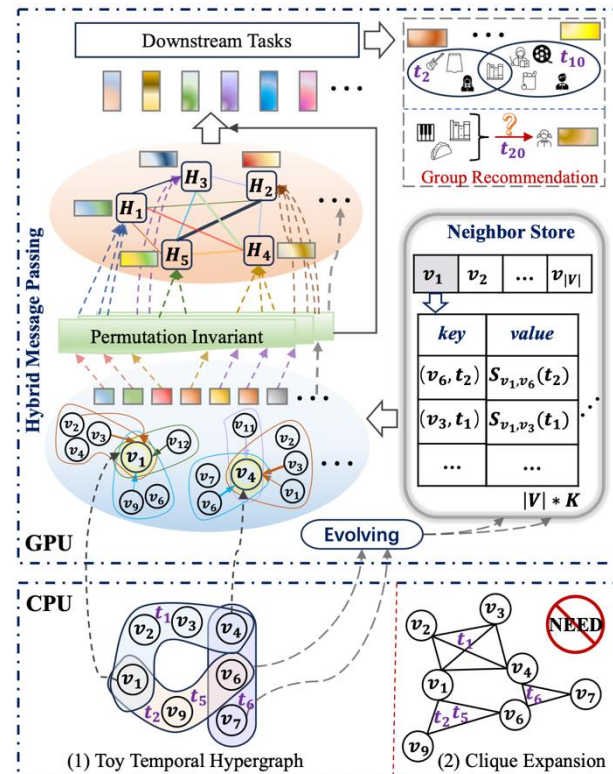
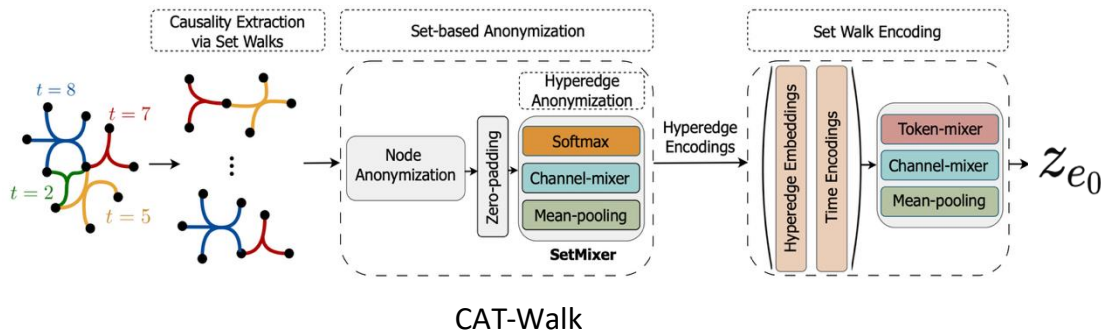
Comparison FastHeP with CAT-Walk

Computation graph complexity

- Hybrid structure learning
- $O(|H_{\mathcal{E}}|K)$ vs. $O(|H_{\mathcal{E}}|^2 K^L)$

Memory cost

- Hyperedge-centric neighbor store
- $O(|\mathcal{V}|)$ vs. $O(|\mathcal{V}||\mathcal{E}|)$



Experimental Setup

- ☐ Baselines
 - Temporal hypergraph approach (CAT-Walk)
 - Static hypergraph approach (NHP, HyperSAGCN, CHESHIRE)
 - CE methods (CE-CAW, CE-Zebra, and CE-Orca)
- ☐ Eight datasets
- ☐ Downstream tasks
 - Hyperedge prediction
- ☐ Transductive and inductive settings

Alias	Dataset	$ \mathcal{V} $	$ \mathcal{E} $	#Timestamps	$\max H_{\mathcal{E}} $
EE	Email-Enron	143	10,883	10,788	18
NC	NDC-Classes	1,161	49,724	5,891	24
NS	NDC-Substances	5,311	112,405	7,734	25
UT	Users-Threads	125,602	192,947	189,917	14
CB	Congress-Bills	1,718	260,851	5,936	25
TMS	Threads-Math-Sx	176,445	719,792	718,340	21
CD	Coauth-DBLP	1,924,991	3,700,067	83	25
TSO	Threads-Stack-Overflow	2,675,955	11,305,343	11,260,218	25

Hyperedge Prediction (cont.)

☐ A transductive setting

Transductive Setting.																
Approach	EE		NC		NS		UT		CB		TMS		CD		TSO	
	AUC	AP	AUC	AP	AUC	AP	AUC	AP	AUC	AP	AUC	AP	AUC	AP	AUC	AP
NHP HyperSAGCN CHESHIRE	63.17	66.87	82.39	80.72	81.38	82.17	82.33	81.44	80.27	77.82	79.20	78.53	-	-	-	-
	<u>83.59</u>	80.54	80.76	80.50	85.07	85.32	85.18	80.99	82.84	81.12	81.77	79.56	-	-	-	-
	82.27	81.39	84.91	82.24	86.30	83.18	82.75	81.96	86.81	83.66	-	-	-	-	-	-
CE-CAW	79.57	78.37	76.30	77.73	84.72	84.93	80.86	80.57	76.99	77.05	80.91	83.40	-	-	-	-
CE-Zebra	80.17	76.29	91.95	91.54	88.99	89.05	92.16	91.03	<u>89.23</u>	87.71	88.02	87.72	-	-	-	-
CE-Orca	78.62	78.35	87.92	87.39	86.54	86.77	83.25	80.13	87.97	87.72	72.61	67.31	-	-	-	-
CATWalk	80.47	<u>82.87</u>	<u>98.72</u>	<u>98.71</u>	<u>90.64</u>	<u>91.96</u>	<u>93.51</u>	<u>94.98</u>	88.15	<u>88.66</u>	<u>90.63</u>	<u>91.26</u>	-	-	-	-
FastHeP	89.93	92.82	99.84	99.01	94.57	93.52	97.44	96.54	98.69	98.51	97.89	96.61	91.38	90.52	93.89	93.78

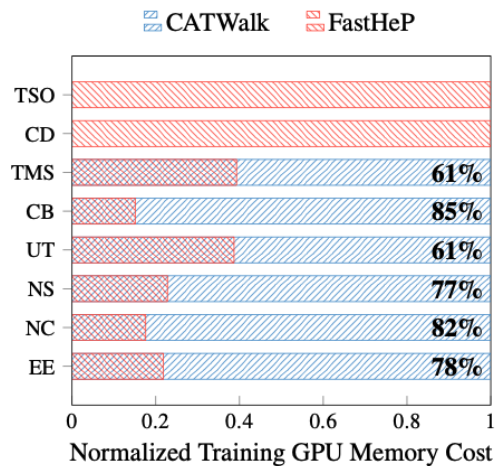
Hyperedge Prediction

☐ Inductive settings

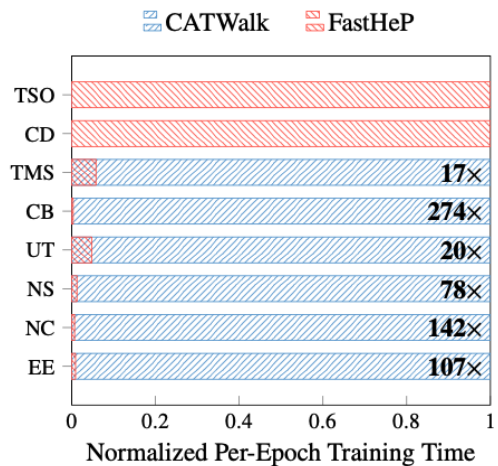
Strongly Inductive Setting.																
Approach	EE		NC		NS		UT		CB		TMS		CD		TSO	
	AUC	AP	AUC	AP	AUC	AP	AUC	AP	AUC	AP	AUC	AP	AUC	AP	AUC	AP
NHP HyperSAGCN CHESHIRE	49.71	50.01	70.53	68.18	70.43	73.22	71.83	71.09	69.82	64.09	68.34	69.07	-	-	-	-
	73.09	72.29	79.05	77.24	75.85	73.24	73.94	72.97	79.51	80.58	75.68	76.01	-	-	-	-
	70.03	72.97	72.24	70.31	76.60	74.91	76.99	74.03	79.43	78.63	-	-	-	-	-	-
CE-CAW	70.81	72.07	76.45	78.58	75.87	73.19	73.12	72.64	75.83	77.19	70.50	69.37	-	-	-	-
CE-Zebra	81.07	<u>83.86</u>	90.51	88.67	92.35	92.71	<u>92.25</u>	90.69	88.06	82.74	86.15	87.33	-	-	-	-
CE-Orca	<u>84.26</u>	82.84	86.82	85.25	87.41	85.99	72.62	69.25	84.03	76.83	69.29	65.13	-	-	-	-
CATWalk	73.45	74.66	<u>98.89</u>	<u>98.97</u>	<u>93.29</u>	<u>94.26</u>	89.84	<u>91.58</u>	<u>93.54</u>	<u>93.93</u>	<u>88.03</u>	<u>88.46</u>	-	-	-	-
FastHeP	90.49	92.30	99.32	99.06	96.68	96.59	96.94	95.85	97.98	98.84	95.22	95.10	92.04	91.25	95.67	95.77
Weakly Inductive Setting.																
Approach	EE		NC		NS		UT		CB		TMS		CD		TSO	
	AUC	AP	AUC	AP	AUC	AP	AUC	AP	AUC	AP	AUC	AP	AUC	AP	AUC	AP
NHP HyperSAGCN CHESHIRE	60.38	55.62	75.17	77.23	73.37	70.62	76.44	75.23	69.58	72.48	70.34	65.71	-	-	-	-
	78.86	<u>79.14</u>	79.45	80.32	80.82	76.67	74.22	70.57	80.12	73.48	78.05	77.39	-	-	-	-
	74.53	75.88	79.03	78.98	80.68	80.78	81.27	82.96	79.67	80.03	-	-	-	-	-	-
CE-CAW	<u>80.54</u>	77.41	77.61	80.03	77.28	79.24	80.79	81.88	79.51	80.39	81.41	83.33	-	-	-	-
CE-Zebra	77.04	75.37	92.38	92.18	90.31	90.88	93.10	92.72	92.45	90.73	88.59	89.20	-	-	-	-
CE-Orca	77.60	77.78	88.76	88.74	85.33	87.68	82.70	79.61	89.79	87.72	73.61	67.92	-	-	-	-
CATWalk	64.11	67.68	<u>99.16</u>	<u>99.33</u>	<u>95.65</u>	<u>96.18</u>	<u>97.15</u>	<u>97.55</u>	<u>98.38</u>	<u>98.48</u>	<u>89.89</u>	<u>90.05</u>	-	-	-	-
FastHeP	90.07	93.95	99.58	99.53	97.97	97.45	98.57	98.78	99.06	98.68	96.19	96.56	93.33	93.01	94.35	95.01

Overall Efficiency Evaluation

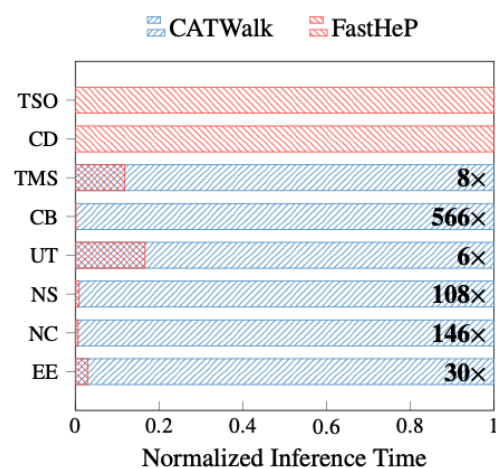
- More efficient than CAT-Walk
- Scalability over large temporal hypergraphs



(a) Training GPU memory cost



(b) Per-epoch training time



(c) Inference time

☐ Existing Models, CAT-Walk

- A hyperedge-centric random walk on hypergraphs, Set Walk
- Causing sub-optimal embedding quality
- Incurring prohibitive computational complexity

☐ FastHeP

- Online hyperedge-centric neighbor store
- A hybrid message passing
- Learning local high-order structures, fusing them to model global high-order correlations

Thank you